

7 AI Prompts Every PM Should Have Saved.

Copy. Paste. Get your time back.

These are the prompts I actually use as a Department Manager in medtech product. Not the ones I'd write for a course.

Each one is longer than it looks because context is what makes AI useful.

Replace the brackets. Run it. Adjust.

WHAT'S INSIDE:

- 01 Write user stories
- 02 Prioritize my backlog (RICE)
- 03 Functional requirements devs won't question
- 04 Write a stakeholder update
- 05 Explain this technical concept
- 06 Run my sprint retro
- 07 Prepare for a hard conversation

NOTE:

Each prompt ends with "What to ask next" — a follow-up that takes the output further.

WHEN TO USE:

You're starting a new feature and need dev-ready stories — fast.

WHY THIS WORKS:

Generic prompts get generic stories. The more context you give about your users and product, the more specific and usable the output.

PROMPT — COPY & PASTE

Write user stories for **[feature name]** in my product **[product description]**.

My users are: **[describe user type, e.g. junior PMs using it for the first time]**.

For each story use this exact format:

As a **[user type]**, I want **[specific action]**, so that **[concrete benefit]**.

After the stories, give me:

- Acceptance criteria (3-5 per story, testable, binary pass/fail)
- Edge cases I might have missed
- Priority ranking with one sentence rationale per story
- Flag any story that seems too large for one sprint

WHAT TO ASK NEXT — take the output further:

"Which of these stories has the most technical uncertainty?"

What questions should I ask the dev team before committing to the sprint?"

PRO TIP:

The more you describe the user's context and pain point, the better.

Vague user = vague story = blocked sprint.

WHEN TO USE:

Your backlog is a mess and you need to defend prioritization to stakeholders.

WHY THIS WORKS:

RICE scoring without rationale is just numbers. This prompt forces AI to explain its reasoning — so you can challenge or defend it.

PROMPT — COPY & PASTE

Here's my backlog: **[list your features or tasks, one per line]**.

Score each item using RICE:

- Reach: how many users affected per quarter
- Impact: effect on the goal (0.25 / 0.5 / 1 / 2 / 3)
- Confidence: how sure are we (100% / 80% / 50%)
- Effort: person-months to build

Ask me clarifying questions first if any item is unclear.

Then give me a ranked table: item, RICE score, one-line rationale.

Flag anything that seems like a quick win vs. a strategic bet.

WHAT TO ASK NEXT — take the output further:

"Which of the top 3 items carries the most risk if we get it wrong?
What should I validate before committing to each one?"

PRO TIP:

Paste 6–10 items minimum. With fewer, the contrast between scores disappears.
That contrast is the whole point.

WHEN TO USE:

You need to hand off a feature without getting 10 follow-up questions.

WHY THIS WORKS:

Most requirements fail at interpretation, not at writing. This prompt forces structure that eliminates ambiguity before it becomes a blocker.

PROMPT — COPY & PASTE

Write a functional requirement for **[feature name]**.

Include all of the following sections:

1. Purpose — why this feature exists and what problem it solves
2. User flow — step by step, every screen state and decision point
3. Inputs and outputs — what data goes in, what comes out
4. Edge cases — what happens when things go wrong or unexpected
5. Acceptance criteria — binary, testable, no interpretation needed

Audience: software developers.

Style: precise, zero ambiguity. If something is unclear, ask me.

Context: **[describe the feature, the product, and who uses it]**

WHAT TO ASK NEXT — take the output further:

"What questions might a developer still have after reading this?"

Rewrite the sections that leave room for interpretation."

PRO TIP:

After it generates, run the follow-up before you share with devs.

It catches 80% of the gaps in one extra step.

WHEN TO USE:

You need to report to leadership without sounding like you're guessing.

WHY THIS WORKS:

Stakeholders don't want details — they want signal, risk, and decisions.

This prompt structures updates so leadership reads them in 60 seconds.

PROMPT — COPY & PASTE

Write a stakeholder update for **[project name]**.

Use this exact structure:

- Status: **[on track / at risk / delayed]** and one sentence why
- Done this week: **[what was completed — be specific, not vague]**
- Next 7 days: **[what's coming and who owns it]**
- Risks and blockers: **[anything that could slow us down]**
- Decisions needed: **[exactly what you need from stakeholders]**

Length: max 150 words. Tone: confident and transparent.

Avoid: vague progress language, passive voice, hedging.

WHAT TO ASK NEXT — take the output further:

"Rewrite the risks section to be more specific about impact.

What happens if each risk materializes and by when?"

PRO TIP:

'Decisions needed' is the most important field.

If you leave it blank, stakeholders don't know why they're reading this.

WHEN TO USE:

A dev said something in a meeting and you need to catch up fast.

WHY THIS WORKS:

PMs don't need to know how to build it — they need to know enough to make good decisions. This prompt targets exactly that level.

PROMPT — COPY & PASTE

Explain **[technical term, concept, or architecture decision]** to me.

I am a product manager. I need to understand this well enough to make product decisions — not to build or implement it myself.

Structure your explanation like this:

1. What it is — in plain language, no jargon
2. Why it matters for the product — impact on users or dev velocity
3. What trade-offs or risks come with this approach
4. What questions I should ask the dev team about this
5. What a bad decision around this looks like — so I can avoid it

WHAT TO ASK NEXT — take the output further:

"What are common mistakes product managers make when dealing with this? Give me a real example of one that went wrong."

PRO TIP:

Follow up with the mistakes question every time.
It's faster than learning from your own experience.

WHEN TO USE:

You're facilitating retro and want structure, not blank slides.

WHY THIS WORKS:

Running retro without prep leads to venting sessions with no outcomes.

This prompt turns your raw answers into actual action items.

PROMPT — COPY & PASTE

Facilitate a sprint retrospective with me. Ask me questions one at a time.

Wait for my full answer before asking the next question.

Step 1 — What went well this sprint? (ask me this first)

Step 2 — What didn't go well or slowed us down?

Step 3 — What one thing would make next sprint better?

Step 4 — Are there any blockers nobody is talking about openly?

After I answer all questions:

- Summarize the 3 key insights from my answers
- Give me 3 concrete action items with a suggested owner and deadline
- Flag any systemic pattern you notice across my answers

WHAT TO ASK NEXT — take the output further:

"Turn the action items into Jira-ready tasks with acceptance criteria.

Write them so a dev understands what done looks like."

PRO TIP:

Run this solo the evening before retro.

You'll walk in with insights — not just questions.

WHEN TO USE:

You need to push back, deliver bad news, or ask for more resources.

WHY THIS WORKS:

Hard conversations fail because of weak openers and no plan for pushback.

This prompt builds the full structure — not just talking points.

PROMPT — COPY & PASTE

Help me prepare for a difficult conversation with **[role/title]**.

Context:

- The issue: **[describe the situation clearly and honestly]**
- My goal: **[what outcome I want from this conversation]**
- Their likely position: **[what they'll push back on and why]**
- What I'm afraid of: **[what could go wrong in this conversation]**

Give me:

1. An opening line — direct, not apologetic
2. My 3 key points with supporting rationale for each
3. Their top 2 likely objections and how I respond to each
4. How to close — what I ask for specifically at the end

WHAT TO ASK NEXT — take the output further:

"Role-play this conversation with me. You play [their role].

Push back the way they would. I'll respond and you coach me after."

PRO TIP:

Run this 30 minutes before — not the night before.

Fresher in your head when it matters.



ALL 7 SAVED

You're set.

These prompts work. But they work better the more context you give. The brackets are the minimum.

I'm still learning. Still adjusting. If one saves you 30 minutes this week — that's enough.

THIS IS JUST THE START.

On @drivetheproduct I post the full process behind these prompts — what to do with the output, how to adjust when it's off, and what I've learned from using AI in a regulated medtech environment.

FOLLOW FOR MORE REAL PM CONTENT

@drivetheproduct

Instagram · LinkedIn: Rafal Bedkowski